17. Idiomatic Go: Effective Patterns

17. Idiomatic Go: Effective Patterns

Phase: Phase 3 — Idioms and engineering judgment
Estimated time: 1–2 days
Slug: `17-idiomatic-go`

Learning objectives

- Apply Effective Go naming and package heuristics
- Make the zero value useful
- Keep interfaces small and local
- Write clear Go that reviewers trust

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Idiomatic Go is a shared dialect: clarity, small interfaces, explicit errors. Teams move faster when style is predictable.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Accept interfaces, return structs.
- Make the zero value useful when possible.
- Short names for short scopes; longer names for package-level.
- Handle errors once; don't log-and-return repeatedly.
- gofmt/goimports are non-negotiable — style is automated.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-idiomatic && cd /tmp/gw-idiomatic
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import (
    "fmt"
    "strconv"
)

func ParsePort(s string) (int, error) {
    n, err := strconv.Atoi(s)
    if err != nil {
        return 0, fmt.Errorf("parse port: %w", err)
    }
    if n < 1 || n > 65535 {
        return 0, fmt.Errorf("port %d out of range", n)
    }
    return n, nil
}

func main() {
    fmt.Println(ParsePort("8080"))
}
```

Run it:

```
go run .
```

Expected output:

```
8080 <nil>
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Refactor a nested if pyramid into early returns.
2. Rename variables to Go-ish short forms inside a small func.
3. Run gofmt on a deliberately messy file.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Names

- Packages: short, lowercase, no underscores (`http`, `json`)
- Interfaces: often `-er` (`Reader`) when one method
- Avoid stuttering: `http.Server` not `http.HTTPServer` in the package name sense

Useful zero values

`var buf bytes.Buffer` works immediately. Design types so `var t T` is ready or clearly invalid.

Return early

Flatten happy path; handle errors first. Avoid deep nesting.

Don’t communicate by sharing memory…

…share memory by communicating (channels) — but also learn mutexes (later). Idiomatic means choosing the right tool, not dogma.

Failure modes to watch while coding

- Java-style getters for every field
- Huge package “utils” dumping grounds
- Interface pollution (exporting interfaces no one implements outside)

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- PEP8 vs gofmt — Go ends style debates with a tool.
- EAFP in Python vs explicit err checks in Go.
- Classes everywhere vs funcs + small structs.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Read Effective Go and the Google Go Style Guide once; then practice by reviewing your own PRs.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Idiomatic Go: Effective Patterns” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Java-style getters for every field
- Huge package “utils” dumping grounds
- Interface pollution (exporting interfaces no one implements outside)

Exercises

Work in `exercises/17-idiomatic-go/`. Read that folder’s `README.md` before you code.

What you will build

Implement the idiomatic parse helper so tests pass with clear errors.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/17-idiomatic-go/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What is a useful zero value example?
2. Why short package names?
3. When is stuttering bad?

Further reading

- https://go.dev/doc/effective_go
- https://go.dev/wiki/CodeReviewComments